GPT-OSS: Model Implementation

Mixture of Experts, Attention Sinks, Sliding Window, YaRN RoPE

1

Where We Are

- ✅ The original Transformer (Vaswani et al., 2017) — attention, FFN, residuals, layer norm, sinusoidal PE
- ✅ Modern transformer upgrades — RMSNorm, RoPE, SwiGLU, GQA, KV-Cache, Flash Attention
- 📍 Today: Walk through `gpt_oss/torch/model.py`
- **Goal**: After this lecture, you can read `model.py` top-to-bottom and understand almost everything in it
- **What's new**: We have seen the individual upgrades in isolation. Today we see how they compose in a real codebase, plus several things we didn't cover:
- • Mixture of Experts (MoE) — sparse expert routing replaces the dense FFN
- • Attention sinks — a learnable "null target" that absorbs unused attention mass
- • Sliding window attention — alternating layers between local and global attention
- • YaRN RoPE — the specific NTK-by-parts context extension algorithm
- • No batch dimension — the model processes a single sequence at a time
- • No KV-cache — this reference implementation recomputes from scratch each step
- • Tensor-parallel expert sharding — distributing experts across GPUs

2

Part 1: Architecture Overview

3

The Full Model at a Glance

TEXT

```
Token IDs (1D) [n_tokens]
 │
 ▼
Embedding [n_tokens, 2880]
 │
 ▼
┌─ TransformerBlock × 36 ──────────────────────────────┐
│                                                      │
│   ┌─ AttentionBlock ──────────────────────────────┐  │
│   │ RMSNorm → QKV Proj → RoPE(YaRN) →             │  │
│   │ SDPA(sinks, sliding_window) → Out Proj        │  │
│   │ + Residual                                    │  │
│   └───────────────────────────────────────────────┘  │
│                                                      │
│   ┌─ MLPBlock (MoE) ──────────────────────────────┐  │
│   │ RMSNorm → Router(top-4 of 128 experts) →      │  │
│   │ Expert MLP1 → SwiGLU → Expert MLP2 →          │  │
│   │ Weighted Sum + Residual                       │  │
│   └───────────────────────────────────────────────┘  │
│                                                      │
└──────────────────────────────────────────────────────┘
 │
 ▼
Final RMSNorm [n_tokens, 2880]
 │
 ▼
Unembedding (Linear) [n_tokens, 201088]
```

What's Different from the "Generic Modern Transformer"?

- GPT-OSS does:
- • Dense SwiGLU FFN → **Mixture-of-Experts** (MoE) with 128 experts, top-4 routing
- • Standard RoPE → **YaRN-extended RoPE** with NTK-by-parts frequency manipulation ($32\times$ scaling)
- • `F.scaled_dot_product_attention` → Custom `sdpa()` with **attention sinks + sliding window mask**
- • Batch dimension (B, T, D) → **No batch dimension** (T, D) — single sequence processing
- • KV-Cache for generation → **No KV-Cache** — full recomputation each step (reference implementation)
- • Separate gate/up projections → **Fused MLP1** with interleaved gate/linear split inside `swiglu()`
- • Standard Swish/SiLU → **Scaled sigmoid** ($\alpha = 1.702$, approximates GELU) + value clamping + $+1$ bias on linear path
- • Weight tying (embedding = LM head) → **No weight tying** — separate embedding and unembedding
- • No biases in attention → **Biases in QKV and output projections** (PyTorch default `bias=True`)
- **Key numbers**: 36 layers, 64 Q-heads, 8 KV-heads, head_dim=64, hidden=2880, 128 experts (4 active), vocab=201K

5

Part 2: ModelConfig — Every Hyperparameter

6

ModelConfig: The Blueprint

PYTHON

```
@dataclass
class ModelConfig:
    num_hidden_layers: int = 36       # Number of TransformerBlocks
    num_experts: int = 128            # Total experts per MoE layer
    experts_per_token: int = 4        # Active experts per token (t
    vocab_size: int = 201088          # Tokenizer vocabulary size
    hidden_size: int = 2880           # Residual stream width (d_mo
    intermediate_size: int = 2880     # Expert FFN intermediate dim
    swiglu_limit: float = 7.0         # Activation clamping thresho
    head_dim: int = 64                # Dimension per attention hea
    num_attention_heads: int = 64     # Query heads (total)
    num_key_value_heads: int = 8      # KV heads (GQA groups)
    sliding_window: int = 128         # Local attention window (tok
    initial_context_length: int = 4096  # Training context length (
    rope_theta: float = 150000.0      # RoPE base frequency
    rope_scaling_factor: float = 32.0 # YaRN context extension fact
    rope_ntk_alpha: float = 1.0       # NTK-by-parts lower bound
    rope_ntk_beta: float = 32.0       # NTK-by-parts upper bound
```

ModelConfig: Derived Dimensions

- Several important dimensions are computed from the config, not stored directly:
- **GQA group size** = num_attention_heads / num_key_value_heads = $64/8 = 8$ Q-heads per KV-head
- **QKV projection dim** = head_dim $\times$ (num_attention_heads + $2\times$ num_key_value_heads) = $64 \times (64 + 16) = 5120$
- → Larger than hidden_size (2880)! The attention space is wider than the residual stream
- **Attention output dim** = head_dim $\times$ num_attention_heads = $64 \times 64 = 4096$
- → Output projection: $4096 \rightarrow 2880$ (compresses back to residual stream width)
- **MoE MLP1 output** = intermediate_size $\times 2 = 5760$ per expert (gate + linear paths interleaved)
- → After SwiGLU: $5760 \rightarrow 2880$ (halved by the gating split)
- **Effective context** = initial_context_length $\times$ rope_scaling_factor = $4096 \times 32 = 131{,}072$ tokens (~131K)
- Note: intermediate_size == hidden_size (both 2880) — unusual for a dense model, but with 128 experts the total MoE capacity is enormous
- Total expert params per layer: $128 \times (5760 \times 2880 + 5760 + 2880 \times 2880 + 2880) \approx 3.19$ billion — but only 4/128 active per token

8

Part 3: RMSNorm — Quick Review

9

RMSNorm in GPT-OSS: Two Implementation Details

- You already know RMSNorm:
- $$\text{RMSNorm}(x) = \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}} \cdot \gamma$$
- The GPT-OSS implementation adds two precision details worth noting:
- **1. Float32 upcast**: The computation is done in float32 regardless of input dtype (bfloat16)
- `t, dtype = x.float(), x.dtype` — upcasts to float32 for the norm computation
- `return (t * self.scale).to(dtype)` — downcasts the result back to the original dtype
- Why? Squaring and taking means in bfloat16 can overflow or lose precision — the denominator needs accuracy
- **2. Scale parameter in float32**: `self.scale` is stored as `torch.float32` even though the model uses bfloat16
- This is a common pattern: keep normalization parameters in full precision for training stability
- **Where RMSNorm appears**: Before attention (in AttentionBlock), before MoE (in MLPBlock), and after the last layer (final norm in Transformer) — the standard pre-norm pattern
- RMSNorm is used $2 \times 36 + 1 = 73$ times in the full model

10

RMSNorm: The GPT-OSS Implementation

PYTHON

```
class RMSNorm(torch.nn.Module):
    def __init__(
        self, num_features: int, eps: float = 1e-05, device: torch.
    ):
        super().__init__()
        self.num_features = num_features
        self.eps = eps
        self.scale = torch.nn.Parameter(
            torch.ones(num_features, device=device, dtype=torch.flo
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        assert x.shape[-1] == self.num_features
        t, dtype = x.float(), x.dtype       # ① Upcast input to flo
        t = t * torch.rsqrt(                 # ② rsqrt = 1/sqrt (fu
            torch.mean(t**2, dim=-1, keepdim=True)  # ③ RMS² over l
            + self.eps
        )
        return (t * self.scale).to(dtype)    # ④ Scale by γ, downca

# Shape trace (no batch dim!):
# Input x:              (n_tokens, 2880) in bfloat16
# t = x.float():        (n_tokens, 2880) in float32
# mean(t², dim=-1, keepdim=True): (n_tokens, 1)
# rsqrt(...):           (n_tokens, 1)
# t * rsqrt:            (n_tokens, 2880) — broadcasted
# * self.scale:         (n_tokens, 2880) × (2880,) → (n_tokens, 288
# .to(dtype):           back to bfloat16
```

Part 4: YaRN RoPE — Context Length Extension

12

Why YaRN? The Context Extension Problem

- Recall: RoPE encodes position by rotating Q and K vectors, with frequencies $\theta_i = \text{base}^{2i/d}$
- The rotation angle at position m for dimension i is $m/\theta_i = m \cdot \text{base}^{-2i/d}$
- **Problem**: A model trained at 4,096 tokens needs to handle 131K+ tokens at inference
- **Naive approach**: Just feed longer sequences → attention scores become erratic at unseen positions because the rotation angles extrapolate beyond what was seen during training
- **Simple interpolation**: Scale all positions by $1/s$ where s is the scaling factor → all frequencies slow down equally → loses fine-grained local position information
- **YaRN** (Yet another RoPE extensioN, Peng et al. 2023) solves this with **NTK-by-parts**:
- **Insight**: Not all RoPE frequencies need the same treatment
- High-frequency dimensions (small i) — encode fine, local position differences → should **not** be interpolated (extrapolation is fine)
- Low-frequency dimensions (large i) — encode coarse, global position → **must** be interpolated to avoid extrapolation
- Middle frequencies — smooth blend between interpolation and extrapolation
- YaRN also scales the attention logits by a **concentration factor** to compensate for the changed frequency spectrum
- GPT-OSS config: `scaling_factor=32` extends 4K context to 128K tokens

13

YaRN: Three Frequency Regions (NTK-by-Parts)

- The algorithm divides the $d/2$ frequency dimensions into three regions:
- **Region 1 — Extrapolation** (high frequency, low i):
- These frequencies already have short wavelengths → positions within the original context are fine
- Keep original: $f_i^{\text{out}} = 1/\theta_i$ (no change)
- **Region 2 — Interpolation** (low frequency, high i):
- These frequencies have long wavelengths that would extrapolate beyond training range
- Scale down: $f_i^{\text{out}} = 1/(s \cdot \theta_i)$ where s = scaling_factor
- **Region 3 — Blend** (middle frequencies):
- Smooth ramp between interpolation and extrapolation
- Uses a linear ramp: `mask = 1 - clamp((i - low) / (high - low), 0, 1)`
- `inv_freq = interpolation × (1 - mask) + extrapolation × mask`
- **Boundary computation**: low and high thresholds derived from ntk_alpha (α), ntk_beta (β), base, and initial context length
- $$\text{low} = \frac{d}{2} \cdot \frac{\ln(\text{ctx_len}/(\beta \cdot 2\pi))}{\ln(\text{base})} \qquad \text{high} = \frac{d}{2} \cdot \frac{\ln(\text{ctx_len}/(\alpha \cdot 2\pi))}{\ln(\text{base})}$$
- **Concentration factor**: $0.1 \cdot \ln(s) + 1.0$ — scales cos/sin to sharpen attention at extended lengths

YaRN: Computing Inverse Frequencies

PYTHON

```
def _compute_concentration_and_inv_freq(self) -> torch.Tensor:
    """See YaRN paper: https://arxiv.org/abs/2309.00071"""
    # Base frequencies: θ_i = base^(2i/d) for i in [0, d/2)
    freq = self.base ** (
        torch.arange(0, self.head_dim, 2, dtype=torch.float, device
        / self.head_dim
    )  # shape: (d/2,) = (32,) for head_dim=64

    if self.scaling_factor > 1.0:
        concentration = (
            0.1 * math.log(self.scaling_factor) + 1.0
        )  # YaRN concentration  (≈ 1.347 for factor=32)

        d_half = self.head_dim / 2  # = 32
        # NTK by parts — compute boundary indices for the three reg
        low = (
            d_half
            * math.log(self.initial_context_length / (self.ntk_beta
            / math.log(self.base)
        )
        high = (
            d_half
            * math.log(self.initial_context_length / (self.ntk_alph
            / math.log(self.base)
        )
        assert 0 < low < high < d_half - 1  # Must be valid indices

        interpolation = 1.0 / (self.scaling_factor * freq)  # Scale
        extrapolation = 1.0 / freq                          # Origi

        # Linear ramp: 0 below 'low', 1 above 'high', linear blend
        ramp = (
            torch.arange(d_half, dtype=torch.float32, device=freq.d
        ) / (high - low)
        mask = 1 - ramp.clamp(0, 1)  # 1 → extrapolate, 0 → interpo

        # Blend: low-i dims use extrapolation, high-i dims use inte
        inv_freq = interpolation * (1 - mask) + extrapolation * mas
    else:
        concentration = 1.0
        inv_freq = 1.0 / freq  # Standard RoPE
```

```
    return concentration, inv_freq
```

YaRN: Computing cos/sin and Applying RoPE

PYTHON

```
# --- RotaryEmbedding methods ---
def _compute_cos_sin(self, num_tokens: int):
    concentration, inv_freq = self._compute_concentration_and_inv_f
    t = torch.arange(num_tokens, dtype=torch.float32, device=self.d
    freqs = torch.einsum("i,j->ij", t, inv_freq)  # Outer product:
    cos = freqs.cos() * concentration               # Scale by YaRN
    sin = freqs.sin() * concentration
    return cos, sin  # Both shape: (num_tokens, d/2)

def forward(self, query, key):
    num_tokens = query.shape[0]
    cos, sin = self._compute_cos_sin(num_tokens)  # (T, 32)

    query_shape = query.shape                      # Save original:
    query = query.view(num_tokens, -1, self.head_dim)  # Flatten he
    query = _apply_rotary_emb(query, cos, sin)
    query = query.reshape(query_shape)             # Restore: (T, 8

    key_shape = key.shape                          # Save original:
    key = key.view(num_tokens, -1, self.head_dim)  # (T, 8, 64) — u
    key = _apply_rotary_emb(key, cos, sin)
    key = key.reshape(key_shape)                   # Restore: (T, 8
    return query, key

# --- The rotation itself (module-level function) ---
def _apply_rotary_emb(x, cos, sin):
    cos = cos.unsqueeze(-2).to(x.dtype)  # (T, 1, d/2) — broadcast
    sin = sin.unsqueeze(-2).to(x.dtype)
    x1, x2 = torch.chunk(x, 2, dim=-1)  # Split last dim in half: e
    o1 = x1 * cos - x2 * sin            # 2D rotation formula
    o2 = x2 * cos + x1 * sin
    return torch.cat((o1, o2), dim=-1)   # Reassemble: (T, H, d)

# This is the "split-half" RoPE variant (not interleaved pairs).
```

Part 5: Custom Attention — Sinks and Sliding Window

17

Attention Sinks: Why They Exist

- **The problem**: In standard softmax attention, every query must distribute 100% of its probability mass across the keys
- But sometimes a query position has nothing useful to attend to — early tokens with few predecessors, or tokens in a region with no relevant context
- In this case, the model is **forced** to spread attention mass over irrelevant keys → noisy, low-quality aggregation
- **Observation** (Xiao et al., 2023 — StreamingLLM): In trained models, the first token (often BOS) absorbs disproportionate attention mass — it acts as an "attention sink"
- The first token's actual content is irrelevant — the model just needs somewhere to dump unused attention probability
- **GPT-OSS solution**: Instead of relying on a specific token, add a **learnable sink logit per attention head**
- Each head has one scalar parameter s_h — appended as an extra column to the attention logits before softmax
- After softmax, some probability goes to the sink → the remaining attention weights sum to ≤ 1
- The sink column is then **removed** — no value vector is associated with it
- **Effect**: The model can "waste" attention probability cleanly, reducing noise in the attention output
- This is a learnable per-head parameter (64 scalars total) — essentially zero overhead

Sliding Window Attention: Local vs Global

- **Full causal attention**: Every token attends to all previous tokens — cost is $O(T^2)$
- **Sliding window attention**: Each token only attends to the most recent W tokens — cost is $O(T \times W)$
- With $W = 128$, token at position 200 can attend to positions 73–200 (128 positions back)
- **Why use it?** Most relevant context is local — nearby tokens matter most for syntax, coreference, etc.
- But you still need global attention for long-range dependencies (references across paragraphs, etc.)
- **GPT-OSS solution**: Alternate between local and global attention by layer:
- • Even layers (0, 2, 4, ...): Sliding window ($W = 128$) — local attention
- • Odd layers (1, 3, 5, ...): Full causal attention — global attention
- This is set in `AttentionBlock.__init__`:
- `self.sliding_window = config.sliding_window if layer_idx % 2 == 0 else 0`
- (`sliding_window=0` means no window → full causal attention)
- **Benefit**: 50% of layers use $O(T \times 128)$ instead of $O(T^2)$ attention — significant savings at long context
- Information flows globally through the odd layers; local patterns are refined in even layers
- Similar to Mistral's approach (Jiang et al., 2023) and Gemma 2 (Riviere et al., 2024)

19

Quick Reference: Einstein Summation (einsum)

- GPT-OSS uses `torch.einsum` for tensor contractions — if you haven't seen it before, here's the pattern:
- `torch.einsum("subscripts", tensor_A, tensor_B)` — each letter is a dimension label
- • Letters on the left of `->` label the input dimensions
- • Letters on the right of `->` label the output dimensions
- • Letters that appear in inputs but **not** in the output are **summed over** (contracted)
- Common examples:
- `"ij,jk->ik"` — matrix multiply: sum over j (standard matmul)
- `"i,j->ij"` — outer product: no contraction, all dims kept
- `"bi,bi->b"` — batched dot product: sum over i
- In GPT-OSS **attention**: `"qhmd,khmd->hmqk"` means:
- q=query position, k=key position, h=KV head, m=Q-group index, d=head dim (contracted)
- Output has dimensions h, m, q, k — the attention score matrix
- In GPT-OSS **MoE**: `"beck,bk->bec"` means:
- b=token, e=expert index, c=output dim, k=input dim (contracted)
- This is a batched matrix-vector multiply: each token × each expert's weight matrix

20

Custom SDPA: Full Implementation

PYTHON

```
def sdpa(Q, K, V, S, sm_scale, sliding_window=0):
    # sliding_window == 0 means no sliding window
    # Q: (T, n_kv_heads, q_mult, d_head) — q_mult = n_q_heads / n_k
    # K: (T, n_kv_heads, d_head)          — one K per KV-head group
    # V: (T, n_kv_heads, d_head)          — one V per KV-head group
    # S: (n_q_heads,) = (64,)             — attention sink logits
    n_tokens, n_heads, q_mult, d_head = Q.shape  # n_heads = n_kv_h
    assert K.shape == (n_tokens, n_heads, d_head)
    assert V.shape == (n_tokens, n_heads, d_head)

    # ① Broadcast K, V to match Q's group dimension
    K = K[:, :, None, :].expand(-1, -1, q_mult, -1)  # (T, 8, 8, 64
    V = V[:, :, None, :].expand(-1, -1, q_mult, -1)  # (T, 8, 8, 64

    # ② Reshape sinks: one scalar per (kv_head, q_group_member)
    S = S.reshape(n_heads, q_mult, 1, 1).expand(-1, -1, n_tokens, -

    # ③ Causal mask (upper triangle = -inf)
    mask = torch.triu(Q.new_full((n_tokens, n_tokens), -float("inf"
    if sliding_window > 0:  # Add sliding window mask (lower triang
        mask += torch.tril(
            mask.new_full((n_tokens, n_tokens), -float("inf")),
            diagonal=-sliding_window
        )

    # ④ Attention scores + sinks
    QK = torch.einsum("qhmd,khmd->hmqk", Q, K)  # (8, 8, T, T)
    QK *= sm_scale                               # Scale by 1/√d
    QK += mask[None, None, :, :]                 # Apply causal +
    QK = torch.cat([QK, S], dim=-1)              # (8, 8, T, T+1)

    # ⑤ Softmax over T+1 positions (including sink)
    W = torch.softmax(QK, dim=-1)                # (8, 8, T, T+1)
    W = W[..., :-1]                              # (8, 8, T, T) ←
    # Now W sums to ≤ 1 per query (some mass went to the sink)

    # ⑥ Weighted sum of values
    attn = torch.einsum("hmqk,khmd->qhmd", W, V)  # (T, 8, 8, 64)
    return attn.reshape(n_tokens, -1)              # (T, 4096)

# The sink absorbs "unused" attention — after removing the sink col
# attention weights per query sum to ≤ 1.0 instead of exactly 1.0.
```

21

```
# This lets the model gracefully ignore irrelevant context.
```

Part 6: AttentionBlock — Putting It Together

22

AttentionBlock: __init__

PYTHON

```
class AttentionBlock(torch.nn.Module):
    def __init__(self, config: ModelConfig, layer_idx: int, device:
        super().__init__()
        self.config = config
        self.norm = RMSNorm(config.hidden_size, device=device)

        # QKV fused projection: 2880 → 5120
        # = head_dim × (num_attention_heads + 2 × num_key_value_hea
        # = 64 × (64 + 16) = 5120
        self.qkv = torch.nn.Linear(
            config.hidden_size,
            config.head_dim * (config.num_attention_heads + 2 * con
            device=device,
        )

        # Output projection: 4096 → 2880
        self.out = torch.nn.Linear(
            config.head_dim * config.num_attention_heads,  # 64 × 6
            config.hidden_size,                            # 2880
            device=device,
        )

        # Rotary embeddings (YaRN-extended)
        self.rotary_emb = RotaryEmbedding(config, device=device)

        # Attention sinks: one learnable scalar per Q-head
        self.S = torch.nn.Parameter(torch.zeros(config.num_attentio

        # Even layers → sliding window (128); Odd layers → full cau
        self.sliding_window = config.sliding_window if layer_idx %
        self.sm_scale = config.head_dim ** -0.5  # 1/√64 = 0.125
```

AttentionBlock: forward — Shape Trace

PYTHON

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    n_tokens = x.shape[0]  # x: (T, 2880)
    cfg = self.config
    n_q = cfg.num_attention_heads      # 64
    n_kv = cfg.num_key_value_heads     # 8
    d = cfg.head_dim                   # 64
    q_per_kv = n_q // n_kv             # 8 (GQA group size)

    # ① Pre-norm + fused QKV projection
    qkv = self.qkv(self.norm(x))       # (T, 2880) → (T, 5120)

    # ② Slice into Q, K, V
    q = qkv[:, : n_q * d]                          # (T, 4096)
    k = qkv[:, n_q * d : (n_q + n_kv) * d]         # (T, 512)
    v = qkv[:, (n_q + n_kv) * d :]                  # (T, 512)

    # ③ Reshape for GQA: Q gets an extra group dimension
    q = q.view(n_tokens, n_kv, q_per_kv, d)  # (T, 8, 8, 64)
    k = k.view(n_tokens, n_kv, d)             # (T, 8, 64)
    v = v.view(n_tokens, n_kv, d)             # (T, 8, 64)

    # ④ Apply YaRN RoPE to Q and K
    q, k = self.rotary_emb(q, k)      # Shapes unchanged

    # ⑤ Custom SDPA with sinks + sliding window
    attn = sdpa(q, k, v, self.S, self.sm_scale, self.sliding_window

    # ⑥ Output projection + residual connection
    return x + self.out(attn)          # (T, 4096) → (T, 2880), the
```

24

AttentionBlock: Key Observations

- **Fused QKV**: A single linear projection produces Q, K, V concatenated — then sliced apart
- This is more efficient than three separate projections (one large matmul vs three smaller ones)
- **GQA reshape**: Q is reshaped to (T, 8, 8, 64) — the middle dimension (8) is the number of Q-heads per KV-head
- K and V remain (T, 8, 64) — one K and one V per KV-head group
- In SDPA, K and V are broadcast (expanded) to match Q's group dimension
- **RoPE applied after reshape**: The rotary embedding operates on the head_dim dimension (last axis)
- Query is temporarily flattened to (T, 64, 64) for rotation, then reshaped back
- **Pre-norm pattern**: RMSNorm is applied before the QKV projection, not after — standard in modern transformers
- **Residual connection**: Applied to the **original** input x, not the normalized version
- `return x + self.out(attn)` — the residual stream is never normalized
- **Biases present**: Both `self.qkv` and `self.out` use biases (PyTorch Linear default) — some models omit these
- **Attention scale**: $1/\sqrt{d} = 1/\sqrt{64} = 0.125$ — prevents softmax saturation as d grows

25

Part 7: SwiGLU Activation — The Gating Mechanism

26

SwiGLU in GPT-OSS: Not Quite Textbook

- SwiGLU: $\text{SwiGLU}(x) = (\text{Swish}(xW_g) \odot xW_l)$
- GPT-OSS modifies this in several ways:
- **1. Fused gate + linear**: Instead of separate W_g and W_l, a single MLP1 produces both **interleaved**
- Output of MLP1 has shape (..., 5760). The `swiglu()` function splits this in half:
- Odd indices → gate path, Even indices → linear path (using `unflatten` + indexing)
- **2. Scaled sigmoid instead of Swish/SiLU**:
- Standard Swish: $\sigma(x) \cdot x$ where σ is the sigmoid
- GPT-OSS: $\sigma(1.702 \cdot x) \cdot x$ — the constant 1.702 makes it approximate GELU
- (From Ramachandran et al., 2017 — this specific α was found via search)
- **3. Value clamping**: Both gate and linear paths are clamped to $[-\text{limit}, \text{limit}]$ before activation
- `limit = 7.0` — prevents extreme values from causing numerical issues
- **4. +1 bias on linear path**: The linear path gets `+1` added before multiplication
- This means the default (at initialization) passes information through rather than zeroing it out
- $$\text{SwiGLU}(x) = \sigma(1.702 \cdot \text{clamp}(x_g)) \cdot x_g \cdot (\text{clamp}(x_l) + 1)$$

SwiGLU: The GPT-OSS Implementation

PYTHON

```
def swiglu(x: torch.Tensor, limit: float) -> torch.Tensor:
    # x shape: (..., intermediate_size * 2)  e.g., (T, 4, 5760)
    # Split into gate and linear paths via interleaving
    x = x.unflatten(-1, (-1, 2))     # (..., 2880, 2)  — reshape la
    x0 = x[..., 0]                   # (..., 2880) — gate path (eve
    x1 = x[..., 1]                   # (..., 2880) — linear path (o

    x0 = x0.clamp(-limit, limit)     # Clamp gate to [-7.0, 7.0]
    x1 = x1.clamp(-limit, limit)     # Clamp linear to [-7.0, 7.0]

    return (
        x0
        * torch.sigmoid(x0 * 1.702)  # Scaled sigmoid ≈ GELU gate
        * (x1 + 1)                   # Linear path with +1 bias
    )

# Shape trace:
# Input:  (T, 4, 5760)   — from MLP1 (per expert, top-4)
# unflatten: (T, 4, 2880, 2)  — split interleaved gate/linear
# x0, x1: (T, 4, 2880)   — each is half the intermediate_size
# Output: (T, 4, 2880)   — ready for MLP2
#
# Note: x0 appears twice — both as the sigmoid input AND
# multiplied by the sigmoid output. This is the "Swish" pattern:
# Swish(x) = x · σ(αx)
```

28

Part 8: Mixture of Experts (MoE) — The MLPBlock

29

MoE: Why Sparse Experts?

- **Dense FFN**: Every token uses the same W_1, W_2 matrices — all parameters active for every token
- **Mixture of Experts**: Replace the single FFN with N independent "expert" FFNs
- A **router** (learned linear layer) scores each expert for each token
- Only the **top-k** experts are activated per token — the rest contribute nothing
- **Why?** Scale model capacity without proportionally scaling compute:
- GPT-OSS has 128 experts × ~25M params each = ~3.2B params per MoE layer
- But only 4 experts fire per token → effective compute ≈ 4 × 25M = ~100M per token per layer
- This is a $32\times$ capacity-to-compute ratio!
- **The router**: A simple linear layer $\mathbb{R}^{2880} \to \mathbb{R}^{128}$ (no bias)
- Softmax gives probabilities → top-4 selected → weights renormalized to sum to 1
- **Expert weights**: Stored as 3D tensors indexed by expert ID
- `mlp1_weight`: (128, 5760, 2880) — each expert maps $2880 \to 5760$
- `mlp2_weight`: (128, 2880, 2880) — each expert maps $2880 \to 2880$
- Plus biases for each layer
- These are **not** `nn.Linear` — they're raw `nn.Parameter` tensors operated on via `torch.einsum`

30

MLPBlock: __init__ — Expert Parameters

PYTHON

```
class MLPBlock(torch.nn.Module):
    def __init__(self, config: ModelConfig, device: torch.device |
        super().__init__()
        self.config = config
        self.norm = RMSNorm(config.hidden_size, device=device)  # P

        # Router: linear layer, no bias, scores each of 128 experts
        self.gate = torch.nn.Linear(
            config.hidden_size,     # 2880
            config.num_experts,     # 128
            device=device,
        )

        # Expert parameters — raw tensors, not nn.Linear!
        E = config.num_experts           # 128
        I = config.intermediate_size * 2 # 5760 (×2 for interleaved
        H = config.hidden_size           # 2880

        # MLP1: up-projection per expert (2880 → 5760)
        self.mlp1_weight = torch.nn.Parameter(torch.empty(E, I, H,
        self.mlp1_bias   = torch.nn.Parameter(torch.empty(E, I, dev

        # MLP2: down-projection per expert (2880 → 2880)
        self.mlp2_weight = torch.nn.Parameter(torch.empty(E, H, H,
        self.mlp2_bias   = torch.nn.Parameter(torch.empty(E, H, dev

# Note: These are (128, ...) tensors — all experts stored together.
# At forward time, we index into them to extract only the top-4 exp
# This is more memory-efficient than 128 separate nn.Linear modules
# and allows batched operations via einsum.
```

31

MLPBlock: forward — Routing and Expert Computation

PYTHON

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    t = self.norm(x)                    # (T, 2880) — pre-norm

    # ① Route: score all 128 experts per token
    gate = torch.softmax(               # Softmax over experts
        self.gate(t), dim=-1            # (T, 2880) → (T, 128)
    )

    # ② Select top-4 experts per token
    expert_weights, expert_indices = torch.topk(
        gate, self.config.experts_per_token, dim=-1  # → (T, 4) eac
    )
    expert_weights = expert_weights / expert_weights.sum(-1, keepdi

    # ③ Gather selected expert parameters
    mlp1_w = self.mlp1_weight[expert_indices]  # (T, 4, 5760, 2880)
    mlp1_b = self.mlp1_bias[expert_indices]    # (T, 4, 5760)
    mlp2_w = self.mlp2_weight[expert_indices]  # (T, 4, 2880, 2880)
    mlp2_b = self.mlp2_bias[expert_indices]    # (T, 4, 2880)

    # ④ MLP1: up-projection per expert
    t = torch.einsum("beck,bk->bec", mlp1_w, t) + mlp1_b  # (T, 4,

    # ⑤ SwiGLU activation
    t = swiglu(t, self.config.swiglu_limit)               # (T, 4,

    # ⑥ MLP2: down-projection per expert
    t = torch.einsum("beck,bec->bek", mlp2_w, t) + mlp2_b  # (T, 4,

    # ⑦ Weighted combination of expert outputs + residual
    t = torch.einsum("bec,be->bc", t, expert_weights)       # (T, 2
    return x + t  # Residual connection
```

MLPBlock: Detailed Shape Trace

- Let's trace every tensor through the MoE forward pass (assume $T = 512$ tokens):
- **Input**: x is $(512, 2880)$
- ① **Router**: `gate(norm(x))` → $(512, 128)$ — one score per expert per token
- ② **Top-k**: `topk(gate, 4)` → `expert_weights` $(512, 4)$, `expert_indices` $(512, 4)$ of dtype int64
- Renormalize: 4 weights per token sum to 1.0
- ③ **Gather**: Index into $(128, 5760, 2880)$ with $(512, 4)$ indices → $(512, 4, 5760, 2880)$
- This is the most memory-intensive step: $512 \times 4 \times 5760 \times 2880 \approx 34$B elements!
- In practice, PyTorch may use views or advanced indexing to avoid full materialization
- ④ **MLP1 einsum** `"beck,bk->bec"`: Contract over k (hidden=2880)
- $(512, 4, 5760, 2880) \times (512, 2880) \to (512, 4, 5760)$
- ⑤ **SwiGLU**: $(512, 4, 5760) \to (512, 4, 2880)$ — halved by gate/linear split
- ⑥ **MLP2 einsum** `"beck,bec->bek"`: Contract over c (intermediate=2880)
- $(512, 4, 2880, 2880) \times (512, 4, 2880) \to (512, 4, 2880)$
- ⑦ **Weighted sum** `"bec,be->bc"`: Contract over e (experts=4)
- $(512, 4, 2880) \times (512, 4) \to (512, 2880)$
- **Output**: $x + t$ is $(512, 2880)$ — same shape as input

33

Part 9: The Full Transformer

34

TransformerBlock and Transformer: Assembly

PYTHON

```
class TransformerBlock(torch.nn.Module):
    """One layer: attention + MoE, with residual connections."""
    def __init__(self, config: ModelConfig, layer_idx: int, device:
        super().__init__()
        self.attention = AttentionBlock(config, layer_idx, device=d
        self.mlp = MLPBlock(config, device=device)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.attention(x)  # Residual is inside AttentionBlock
        x = self.mlp(x)        # Residual is inside MLPBlock
        return x


class Transformer(torch.nn.Module):
    def __init__(self, config: ModelConfig, device: torch.device |
        super().__init__()
        self.config = config

        # Token embedding: 201088 → 2880
        self.embedding = torch.nn.Embedding(
            config.vocab_size, config.hidden_size, device=device
        )

        # 36 transformer blocks
        self.block = torch.nn.ModuleList([
            TransformerBlock(config, layer_idx=i, device=device)
            for i in range(config.num_hidden_layers)
        ])

        # Final layer norm
        self.norm = RMSNorm(config.hidden_size, device=device)

        # Unembedding: 2880 → 201088 (NOT tied with embedding)
        self.unembed = torch.nn.Linear(
            config.hidden_size, config.vocab_size, device=device
        )

    def forward(self, token_ids: torch.Tensor) -> torch.Tensor:
        x = self.embedding(token_ids)  # (T,) → (T, 2880)
        for block in self.block:       # 36 layers
```

```
        x = block(x)                # (T, 2880) → (T, 2880)
    return self.unembed(self.norm(x))  # (T, 2880) → (T, 201088
```

Transformer: End-to-End Data Flow

- **Input**: `token_ids` — a 1D tensor of shape $(T,)$ containing integer token IDs
- No batch dimension! This model processes one sequence at a time
- **Embedding**: $(T,) \xrightarrow{\text{lookup}} (T, 2880)$ — each token ID maps to a 2880-dim vector
- **36 TransformerBlocks**: Each preserves shape $(T, 2880) \to (T, 2880)$
- Internally: attention writes to residual stream, then MoE writes to residual stream
- Even blocks (0, 2, ..., 34): sliding window attention (local, $W = 128$)
- Odd blocks (1, 3, ..., 35): full causal attention (global)
- **Final RMSNorm**: $(T, 2880) \to (T, 2880)$ — normalizes before unembedding
- **Unembedding**: $(T, 2880) \xrightarrow{\text{linear}} (T, 201088)$ — logits over vocabulary
- This is a separate `nn.Linear` (not tied with the embedding)
- Includes a bias term (201088 parameters) — provides per-token frequency prior
- **Output**: Raw logits — not softmax! The caller decides whether to use softmax or argmax
- **Total parameter count**: ~24.5B parameters (but only ~3.5B active per token due to MoE sparsity)
- Note: `self.block` (singular) is a `ModuleList` — just a naming choice

36

Part 10: Weight Loading — from_checkpoint

37

Weight Loading: Overview

- Weights are stored in a checkpoint on disk — the `from_checkpoint` class method loads them into the model
- **Why not just `torch.load()`?**
- 1. The checkpoint uses **MXFP4** format (4-bit floating point) — needs custom deserialization via a `Checkpoint` class
- 2. Expert parameters need to be **sharded across GPUs** for tensor parallelism
- 3. Parameters must be sliced by rank for distributed inference
- **Loading strategy**:
- ① Create the model with uninitialized parameters on the target device
- ② Open the checkpoint (handles MXFP4 → float16/bfloat16 upcasting)
- ③ For each parameter: copy from checkpoint, applying expert sharding where needed
- **Expert sharding**: The 128 experts' MLP weights are split across `world_size` GPUs
- Each GPU holds a slice of the intermediate dimension:
- `mlp1_weight`: full is $(128, 5760, 2880)$ → each GPU gets $(128, 5760/\mathrm{ws}, 2880)$
- `mlp2_weight`: full is $(128, 2880, 2880)$ → each GPU gets $(128, 2880, 2880/\mathrm{ws})$
- `mlp1_bias`: full is $(128, 5760)$ → each GPU gets $(128, 5760/\mathrm{ws})$
- `mlp2_bias`: stays full $(128, 2880)$ — not sharded (needed complete for addition after all_reduce)

38

from_checkpoint: The Loading Code

PYTHON

```
@classmethod
def from_checkpoint(
    cls, checkpoint_path: str, device: str, world_size: int = 1, ra
) -> "Transformer":
    config = ModelConfig()
    if not isinstance(device, torch.device):
        device = torch.device(device)
    model = cls(config, device=device)  # ① Create model (random pa
    model.eval()                         # ② Set to eval mode

    per_rank = config.intermediate_size * 2 // world_size  # 5760 /
    offset = rank * per_rank                               # Start

    with Checkpoint(checkpoint_path) as cp:  # ③ Open MXFP4 checkpo
        for name, param in model.named_parameters():
            full_tensor = cp[name]          # Load & upcast from ch

            if "mlp1" in name:              # Shard MLP1 along inte
                full_tensor = full_tensor[..., offset : offset + pe
                # mlp1_weight: (128, 5760, 2880) → (..., per_rank,
                # mlp1_bias:   (128, 5760)       → (..., per_rank)
                # The ellipsis handles both weight (3D) and bias (2

            elif "mlp2_weight" in name:     # Shard MLP2 weight alo
                full_tensor = full_tensor[..., offset : offset + pe
                # (128, 2880, 2880) → (128, 2880, per_rank)

            # mlp2_bias is NOT sharded — needed in full after all_r

            try:
                param.data.copy_(full_tensor)
            except Exception as e:
                print(f"Error: {name}: {e} ({param.shape} vs {full_
                raise
    return model
```

39

Expert Parallelism: How Sharding Works

- **Problem**: With 128 experts, MLP weights are enormous — sharding across GPUs is essential
- **Strategy**: Shard along the intermediate (inner) dimension — each GPU computes a partial result
- **MLP1** (up-projection): Weight $(128, 5760, 2880)$ sharded on dim 1 → each GPU holds $(128, 5760/\mathrm{ws}, 2880)$
- Each GPU produces a slice of the intermediate activations
- The `...` (ellipsis) in `full_tensor[..., offset:offset+per_rank, :]` handles both:
- – Weight tensor (3D): slices dim 1 of $(128, 5760, 2880)$
- – Bias tensor (2D): slices dim 1 of $(128, 5760)$
- **MLP2** (down-projection): Weight $(128, 2880, 2880)$ sharded on dim 2 → each GPU holds $(128, 2880, 2880/\mathrm{ws})$
- Each GPU computes a partial matrix-vector product → results summed via **all_reduce**
- `full_tensor[..., offset:offset+per_rank]` slices the last dimension
- **MLP2 bias**: NOT sharded — each GPU holds the full $(128, 2880)$ bias
- Added after all_reduce so the bias is applied once to the complete sum
- **Example** with `world_size=2`:
- GPU 0 gets `mlp1_weight[:, 0:2880, :]`, GPU 1 gets `mlp1_weight[:, 2880:5760, :]`
- GPU 0 gets `mlp2_weight[:, :, 0:1440]`, GPU 1 gets `mlp2_weight[:, :, 1440:2880]`
- After MLP2: `all_reduce(partial_0 + partial_1)` → each GPU has the full output

40

Part 11: Token Generation

41

TokenGenerator: __init__ — Model Initialization

PYTHON

```
class TokenGenerator:
    @torch.inference_mode()  # ← Disables gradient tracking entirel
    def __init__(self, checkpoint_path: str, device: str):
        self.device = device
        self.model = Transformer.from_checkpoint(checkpoint_path, d

    # That's it! The constructor:
    #   1. Loads the model from an MXFP4 checkpoint
    #   2. Moves everything to the specified device (e.g., "cuda:0"
    #   3. @torch.inference_mode() means no autograd overhead durin
    #
    # Note what's NOT here:
    #   - No tokenizer! TokenGenerator works with raw token IDs
    #   - No KV-cache initialization
    #   - No batch size configuration
    #   - Tokenization is handled externally before calling generat
```

42

TokenGenerator: generate — The Autoregressive Loop

PYTHON

```
@torch.inference_mode()
def generate(
    self,
    prompt_tokens: list[int],     # Already-tokenized input
    stop_tokens: list[int],       # Token IDs that signal end-of-ge
    temperature: float = 1.0,     # Sampling temperature (0 = greed
    max_tokens: int = 0,          # 0 means unlimited
    return_logprobs: bool = False,
):
    tokens = list(prompt_tokens)
    num_generated = 0

    while max_tokens == 0 or num_generated < max_tokens:
        # ① Run FULL model on ALL tokens (no KV-cache!)
        input_ids = torch.as_tensor(tokens, dtype=torch.int32, devi
        logits = self.model(input_ids)  # (T, 201088) — logits for
        logits = logits[-1]              # (201088,)  — only last p

        # ② Sample or argmax
        if temperature == 0:            # Greedy decoding
            token = logits.argmax(-1).item()
        else:                           # Temperature sampling
            probs = torch.softmax(logits * (1.0 / temperature), dim
            token = torch.multinomial(probs, 1).item()

        tokens.append(token)
        num_generated += 1

        # ③ Yield the token (and optionally log-prob)
        if return_logprobs:
            log_probs = torch.log_softmax(logits, dim=-1)
            yield token, log_probs[token].item()
        else:
            yield token

        # ④ Stop if we hit a stop token
        if token in stop_tokens:
            break
```

TokenGenerator: Design Decisions

- **No KV-cache**: Every call to `self.model(input_ids)` recomputes all layers for all positions
- Generation cost: step n processes n tokens → total cost is $O(n^2)$ in sequence length
- With KV-cache it would be $O(n)$ — but this is a reference implementation prioritizing simplicity
- **No tokenizer**: The `generate()` method receives and yields raw integer token IDs
- Tokenization/detokenization is the caller's responsibility — clean separation of concerns
- **Temperature scaling**: Uses multiplication by $1/T$ rather than division by T
- `logits * (1.0 / temperature)` — minor optimization, avoids a division
- Temperature = 0 → argmax (greedy); Temperature < 1 → sharper; Temperature > 1 → more random
- **Generator pattern**: Uses Python `yield` — tokens are produced lazily, one at a time
- The caller can process tokens as they arrive (streaming) without waiting for the full sequence
- **max_tokens = 0 means unlimited**: The loop runs until a stop token is hit
- **Stop tokens**: Explicit list (e.g., EOS token ID) — checked after each generated token
- **torch.as_tensor**: Converts the list to a tensor without copying if possible — more efficient than `torch.tensor`
- **@torch.inference_mode()**: Decorator disables autograd — faster and uses less memory than `torch.no_grad()`

Generation Trace: Step by Step

- Let's trace generating 3 tokens from a 5-token prompt:
- **Step 1** (first generated token):
- `tokens = [101, 2003, 1037, 6251, 102]` (5 prompt tokens)
- `model(tokens)` → logits $(5, 201088)$ → take `logits[-1]` → $(201088,)$
- Sample from softmax → token `2612` → `tokens = [101, 2003, 1037, 6251, 102, 2612]`
- `yield 2612`
- **Step 2** (second generated token):
- `tokens = [101, 2003, 1037, 6251, 102, 2612]` (6 tokens — recompute everything!)
- `model(tokens)` → logits $(6, 201088)$ → take `logits[-1]` → $(201088,)$
- Sample → token `1012` → append → `yield 1012`
- **Step 3** (third generated token):
- `tokens = [101, 2003, 1037, 6251, 102, 2612, 1012]` (7 tokens)
- `model(tokens)` → logits $(7, 201088)$ → take `logits[-1]` → $(201088,)$
- Sample → token `102` (stop token!) → `yield 102` → `break`
- **Cost analysis**: We computed $5 + 6 + 7 = 18$ token-positions total
- With KV-cache: $5 + 1 + 1 = 7$ token-positions (cache reuses prior computations)
- The redundant work grows quadratically: for n generated tokens from prompt of length p, total = $np + n(n + 1)/2$

Part 12: End-to-End Architecture Diagram

46

Complete Data Flow: Token to Token

TEXT

```
External Tokenizer (not in model.py)
 │
 ▼
Token IDs: [101, 2003, 1037, 6251, 102]     shape: (5,)
 │
 ▼
┌─ Transformer.forward() ──────────────────────────────────────────┐
│                                                                  │
│  Embedding lookup                                  (5,) → (5, 2880)
│  │                                                              │
│  ▼                                                              │
│  TransformerBlock 0 (sliding window W=128)                      │
│   ├─ AttentionBlock: RMSNorm → QKV(5120) → RoPE → SDPA → Out   │
│   └─ MLPBlock:       RMSNorm → Route(top-4/128) → Experts → Σ  │
│  │                                                              │
│  TransformerBlock 1 (full causal attention)                     │
│   ├─ AttentionBlock: RMSNorm → QKV(5120) → RoPE → SDPA → Out   │
│   └─ MLPBlock:       RMSNorm → Route(top-4/128) → Experts → Σ  │
│  │                                                              │
│  ... (36 blocks total, alternating window/full) ...             │
│  │                                                              │
│  TransformerBlock 35 (full causal)                              │
│  │                                                              │
│  ▼                                                              │
│  Final RMSNorm                                (5, 2880) → (5, 2880
│  │                                                              │
│  ▼                                                              │
│  Unembedding (Linear)                         (5, 2880) → (5, 2010
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
 │
 ▼
Logits[:, -1, :] → softmax → sample → next token ID
```

47

Part 13: Summary

48

What We Covered Today

- **Complete walkthrough of `model.py`** — every class, every forward pass, every shape
- **ModelConfig**: 17 hyperparameters that define the architecture (36 layers, 64 Q-heads, 8 KV-heads, 128 experts)
- **RMSNorm**: Float32 upcast for numerical stability, 73 instances in the model
- **YaRN RoPE**: NTK-by-parts frequency manipulation for $32\times$ context extension (4K → 128K)
- Three frequency regions: extrapolation (high freq), interpolation (low freq), smooth blend (middle)
- Concentration factor ≈ 1.347 scales attention at extended lengths
- **Custom SDPA**: Learnable attention sinks (per-head scalars) + alternating sliding window / full causal
- **GQA**: 64 Q-heads grouped into 8 KV-head groups (8 Q-heads share each KV pair)
- **SwiGLU**: Fused gate+linear with scaled sigmoid ($\alpha = 1.702$), clamping, and $+1$ linear bias
- **Mixture of Experts**: 128 experts, top-4 routing via softmax + topk, expert weights as 3D tensors + einsum
- **Tensor-parallel sharding**: MLP1 sharded on intermediate dim, MLP2 on output dim, MLP2 bias unsharded
- **TokenGenerator**: No KV-cache, no tokenizer, no batch dim — a clean reference implementation
- **Generation**: Autoregressive loop with full recomputation at each step, $O(n^2)$ total cost

Key Takeaways

- **1.** A modern LLM is built from a small number of composable building blocks: embedding, attention, FFN/MoE, normalization, position encoding — but the details of each matter enormously
- **2.** The residual stream $(T, 2880)$ is the backbone — every attention and MoE block reads from it and writes back to it, preserving the shape throughout
- **3.** Mixture of Experts gives massive capacity ($128\times$ more FFN parameters) at constant compute cost per token — the key architectural innovation enabling scaling
- **4.** YaRN RoPE, attention sinks, and sliding windows are practical engineering solutions to real problems: context extension, attention waste, and quadratic cost — not just theoretical improvements
- **5.** This reference implementation prioritizes **readability** over performance: no KV-cache, no batching, no Flash Attention — making it ideal for understanding, but not for deployment
- **6.** The `from_checkpoint` method reveals an important production concern: models are too large for single GPUs, requiring tensor parallelism and careful sharding strategies
- **7.** Understanding every line of `model.py` means you can now: modify the architecture, debug inference issues, add a KV-cache, implement new attention patterns, or port to a different framework

Supplementary Resources

🚀 **Interactive Demo:** yarn-rope.html

🚀 **Interactive Demo:** handout.html

51